

Docker Execution Guide (Data Track + API Mocks Track)

This runbook is for terminal-only execution using Docker.

- Repo root (host): /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
 - Container working mount: /work
 - Image name used below: sdp:latest
-

0) Memory Flow

Data track

Lint -> Generate v1/v2 -> Delta -> SCD2 -> (Optional) Infer/Enrich
-> Generate

Mocks track

Init -> Enrich (optional) -> Lint -> Render -> WireMock

1) One-Time Docker Setup

Step 1: Build image

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
docker build -t sdp:latest .
```

Step 2: Verify image

```
docker images | grep sdp
```

2) Data Track (Docker)

All host paths must be mounted as /work/... inside container.

2.1 Lint + Snapshot generation

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
```

```
docker run --rm -v "$PWD:/work" sdp:latest \
  lint --config /work/config/Acct_bkng.xlsx
```

```
docker run --rm -v "$PWD:/work" sdp:latest \
```

```

generate --config /work/config/Acct_bkng.xlsx \
--output /work/output/snap_v1 \
--default-records 1000 --seed 42

docker run --rm -v "$PWD:/work" sdp:latest \
generate --config /work/config/Acct_bkng.xlsx \
--output /work/output/snap_v2 \
--default-records 1000 --seed 43

```

2.2 Delta execution

```

cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator

docker run --rm -v "$PWD:/work" sdp:latest \
delta --config /work/config/Acct_bkng.xlsx \
--previous /work/output/snap_v1 \
--current /work/output/snap_v2 \
--output /work/output/delta_run

```

Optional selected tables:

```

cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator

docker run --rm -v "$PWD:/work" sdp:latest \
delta --config /work/config/Acct_bkng.xlsx \
--previous /work/output/snap_v1 \
--current /work/output/snap_v2 \
--output /work/output/delta_run_subset \
--tables df_cac_acg_entr df_cash_bookg

```

2.3 SCD2 execution

```

cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator

docker run --rm -v "$PWD:/work" sdp:latest \
scd2 --config /work/config/Acct_bkng.xlsx \
--previous /work/output/snap_v1 \
--current /work/output/snap_v2 \
--output /work/output/scd2_run

```

2.4 AI/ML relationship inference (Docker)

```

cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator

docker run --rm -v "$PWD:/work" sdp:latest \
generate --config /work/config/Creditcard_no_rel.xlsx \
--output /work/output/infer_ml_generate \

```

```

--infer-relationships --method ml --ml-confidence 0.55 --seed 42

docker run --rm -v "$PWD:/work" sdp:latest \
  infer-relationships --config /work/config/Creditcard_no_rel.xlsx \
    --config-output /work/config/inferred_creditcard.yaml \
    --er-output /work/diagrams/inferred_creditcard.mmd

```

Optional feedback recording:

```

cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator

docker run --rm -v "$PWD:/work" sdp:latest \
  record-feedback --inferred /work/config/inferred_creditcard.yaml \
    --reviewed /work/config/inferred_creditcard.reviewed.yaml

```

2.5 LLM-based data features (Docker + LM Studio)

For container-to-host LM Studio on macOS, use: - host.docker.internal

```

cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator

docker run --rm -v "$PWD:/work" \
  -e SDP_LLM_PROVIDER="lm-studio" \
  -e SDP_LLM_BASE_URL="http://host.docker.internal:1234/v1" \
  -e SDP_LLM_MODEL="google/gemma-4-e4b" \
  sdp:latest \
  generate --config /work/config/Creditcard_no_rel.xlsx \
    --output /work/output/infer_llm_generate \
    --infer-relationships --method llm --llm-confidence 0.7 --seed 42

docker run --rm -v "$PWD:/work" \
  -e SDP_LLM_PROVIDER="lm-studio" \
  -e SDP_LLM_BASE_URL="http://host.docker.internal:1234/v1" \
  -e SDP_LLM_MODEL="google/gemma-4-e4b" \
  sdp:latest \
  enrich --config /work/config/Creditcard_no_rel.xlsx \
    --output /work/config/creditcard_enriched.yaml \
    --confidence 0.7

docker run --rm -v "$PWD:/work" sdp:latest \
  generate --config /work/config/creditcard_enriched.yaml \
    --output /work/output/enriched_generate --seed 42

```

2.6 ER generation (Docker)

ER during standard generate

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
```

```
docker run --rm -v "$PWD:/work" sdp:latest \
  generate --config /work/config/Acct_bkng.xlsx \
    --output /work/output/run_with_er \
    --er-diagram --er-format mermaid dot
```

ER as PNG (optional)

PNG rendering needs matplotlib available in the image/runtime. If your current image was built without it, rebuild accordingly.

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
```

```
docker run --rm -v "$PWD:/work" sdp:latest \
  generate --config /work/config/Acct_bkng.xlsx \
    --output /work/output/run_with_er_png \
    --er-diagram --er-format png
```

ER from relationship inference workflow

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
```

```
docker run --rm -v "$PWD:/work" sdp:latest \
  infer-relationships --config /work/config/Creditcard_no_rel.xlsx \
    --config-output /work/config/inferred_creditcard.yaml \
    --er-output /work/diagrams/inferred_creditcard.mmd
```

3) API Mocks Track (Docker)

Reference OpenAPI source: - /work/examples/openapi/Sample_complex.yaml

3.1 General flow (no LLM enrich)

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
```

```
docker run --rm -v "$PWD:/work" sdp:latest \
  mock-init --from /work/examples/openapi/Sample_complex.yaml \
    --output /work/mocks/sample_complex.yaml
```

```
docker run --rm -v "$PWD:/work" sdp:latest \
  mock-lint --config /work/mocks/sample_complex.yaml
```

```
docker run --rm -v "$PWD:/work" sdp:latest \
  mock-render --config /work/mocks/sample_complex.yaml \
    --output /work/stubs/sample_complex \
    --format wiremock,json,postman,pact \
    --examples 5 --seed 42 --match-mode any
```

3.2 LLM-enhanced mocks flow

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
```

```
docker run --rm -v "$PWD:/work" \
  -e SDP_LLM_PROVIDER="lm-studio" \
  -e SDP_LLM_BASE_URL="http://host.docker.internal:1234/v1" \
  -e SDP_LLM_MODEL="google/gemma-4-e4b" \
  sdp:latest \
  mock-init --from /work/examples/openapi/Sample_complex.yaml \
    --output /work/mocks/sample_complex.yaml
```

```
docker run --rm -v "$PWD:/work" \
  -e SDP_LLM_PROVIDER="lm-studio" \
  -e SDP_LLM_BASE_URL="http://host.docker.internal:1234/v1" \
  -e SDP_LLM_MODEL="google/gemma-4-e4b" \
  sdp:latest \
  mock-enrich --config /work/mocks/sample_complex.yaml \
    --output /work/mocks/sample_complex.enriched.yaml
```

```
docker run --rm -v "$PWD:/work" sdp:latest \
  mock-lint --config /work/mocks/sample_complex.enriched.yaml
```

```
docker run --rm -v "$PWD:/work" sdp:latest \
  mock-render --config /work/mocks/sample_complex.enriched.yaml \
    --output /work/stubs/sample_complex \
    --format wiremock,json,postman,pact \
    --examples 5 --seed 42 --match-mode any
```

4) Run WireMock After Docker Render

WireMock runs on host, using generated host folder:

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
java -jar ~/tools/wiremock.jar --root-dir stubs/sample_complex/wiremock --port 8081 --verbose
```

Sanity check:

```
curl http://localhost:8081/__admin/mappings | head
```

5) MCP Server via Docker

5.1 Run manually (for quick test)

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
```

```
docker run --rm -i -v "$PWD:/work" \  
-e SDP_LLM_PROVIDER="lm-studio" \  
-e SDP_LLM_BASE_URL="http://host.docker.internal:1234/v1" \  
-e SDP_LLM_MODEL="google/gemma-4-e4b" \  
sdp:latest mcp
```

transport=stdio staying open is expected; server waits for client messages.

5.2 LM Studio mcp.json entry (Docker command)

```
{  
  "mcpServers": {  
    "synthetic-data-platform-docker": {  
      "command": "docker",  
      "args": [  
        "run",  
        "--rm",  
        "-i",  
        "-v",  
        "/Users/natarajankanakasabapathy/FECTECH/TestDataGenerator:/work",  
        "-e",  
        "SDP_LLM_PROVIDER=lm-studio",  
        "-e",  
        "SDP_LLM_BASE_URL=http://host.docker.internal:1234/v1",  
        "-e",  
        "SDP_LLM_MODEL=google/gemma-4-e4b",  
        "sdp:latest",  
        "mcp"  
      ]  
    }  
  }  
}
```

After saving mcp.json, restart LM Studio.

6) Sample MCP Prompt (LM Studio Chat)

Use the synthetic-data-platform tools and do this end-to-end:

- Source: examples/openapi/Sample_complex.yaml
- Output mock config: mocks/sample_complex.yaml
- Enriched config: mocks/sample_complex.enriched.yaml
- Render output dir: stubs/sample_complex
- Formats: wiremock,json,postman,pact
- examples=5, seed=42, match-mode=any
- LLM provider context: lm-studio at http://localhost:1234/v1 with model google/gemma-4-e4b

Tasks:

- 1) Run mock-init from the OpenAPI source.
- 2) Run mock-enrich (optional step, but do it now).
- 3) Run mock-lint on enriched config.
- 4) Run mock-render with the options above.
- 5) Return a short checklist of what was generated and the exact WireMock start command for m

If a step fails, continue with best-effort fallback and clearly mark failed step.

7) docker-compose Shortcuts (Optional)

```
cd /Users/natarajankanakasabapathy/FECTECH/TestDataGenerator
docker compose build
```

```
docker compose run --rm cli \
  generate --config /work/examples/configs/yaml/02_ecommerce_relationships.yaml \
    --output /work/output/from_compose --default-records 500 --seed 7
```

```
docker compose up streamlit
```

8) Quick Troubleshooting

Invalid JSON / EOF in MCP stream

- Avoid shell wrappers in LM Studio MCP command.
- Prefer direct `docker run ... sdp:latest mcp args in mcp.json`.
- Ensure no extra stdout text is emitted before JSON-RPC.

Container cannot reach LM Studio

- On macOS Docker, use `http://host.docker.internal:1234/v1` (not localhost).

Render ran but WireMock fails to start

- Check `stubs/sample_complex/wiremock` exists on host.
- Re-run `mock-render` with same output path.

No files under host output/mocks/stubs

- Confirm `-v "$PWD:/work"` mount is present in every command.